

# Documentacion completa - Sistema hibrido Dev Muscle

Referencia tecnica + guia usuario + workspace  
Generado: 2026-06-27 16:52

## Esquema híbrido — revisión y aclaración

### ¿Tenemos el esquema híbrido?

Sí. Está implementado y operativo en tu máquina.

### Esquema real (lo que tienes)

|                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CURSOR (IDE – reemplaza VS Code en este setup) <ul style="list-style-type: none"><li>Tab / Agent cloud (créditos, escalación)</li><li>Rules + Skills (memoria)</li><li>Continue.dev (extensión)</li><li>Cline (extensión)</li></ul> |
| OLLAMA (motor local, sin créditos)<br>qwen-coder-32k   1.5b autocomplete   nomic-embed                                                                                                                                              |
| MEMORIA + ORGANIZACIÓN<br>workspace/   AGENTS.md   .cursor/rules/   GitHub                                                                                                                                                          |

### VS Code + Continue + Cline + Ollama vs tu setup

| Documentación genérica | Tu setup real |  
|-----|-----|  
| VS Code como editor | **Cursor** como editor |  
| Continue en VS Code | **Continue en Cursor** ☐ |  
| Cline en VS Code | **Cline en Cursor** ☐ |  
| Ollama local | **Ollama local** ☐ |  
| Cursor aparte | **Cursor integra todo** ☐ |

Conclusión: El esquema es el mismo. Solo cambia el nombre del IDE: usas Cursor en lugar de VS Code. No necesitas ambos.

### Estado de componentes (tu PC)

| Componente | Estado |  
|-----|-----|  
| Cursor IDE | ☐ Instalado |  
| Ollama + modelos | ☐ Configurado |  
| Continue | ☐ Extensión + config.yaml |  
| Cline | ☐ Extensión + Ollama configurado |  
| Dev Muscle Skills | ☐ ~/.cursor/skills/ |

## Flujo de costos

80% tareas → Continue / Cline + Ollama → \$0  
15% tareas → Cursor Tab → créditos plan  
5% tareas → Cursor Agent cloud → créditos (escalación)

Memoria (Rules) reduce el 5% con el tiempo.

## Carpeta local + GitHub

```
C:\Users\claud\dev-projects\  
└─ hybrid-dev-system/      ← repo GitHub  
    ├── workspace/         ← proyectos, docs, assets ordenados  
    ├── docs/              ← documentación técnica  
    ├── skills/            ← backup agentes  
    ├── templates/         ← plantillas nuevos proyectos  
    └─ scripts/            ← setup, configure-cline, etc.
```

URL: <https://github.com/cscspachile-existe/hybrid-dev-system>

## Documentación usuario (empezar)

☐ workspace/docs-usuario/01-empezar-aqui.md

# Plan maestro: Sistema híbrido de desarrollo

## Objetivo

Usar Cursor como IDE principal, con una capa local (Ollama + Continue + Cline) que hace el trabajo repetitivo sin créditos, y escalar a Cursor Agent cloud solo cuando lo local no alcanza.

La "memoria" no entrena modelos — persiste conocimiento en Rules, Skills y AGENTS.md.

## Arquitectura

|                    |                   |                   |        |
|--------------------|-------------------|-------------------|--------|
| CURSOR (IDE)       |                   |                   |        |
| Tab autocomplete   | Agent cloud       | Rules             | Skills |
| Continue.dev       | Cline             |                   |        |
| Chat + Tab local   | Agente Plan/Act   |                   |        |
| OLLAMA (local)     |                   |                   |        |
| qwen2.5-coder:14b  | 1.5b autocomplete | nomiic-embed      |        |
| MEMORIA COMPARTIDA |                   |                   |        |
| AGENTS.md          | .cursor/rules/    | ~/.cursor/skills/ |        |

# Flujo de decisión

| Tarea | Herramienta | Costo |  
|-----|-----|-----|  
| Autocomplete mientras escribes | Continue + Ollama 1.5B | Gratis |  
| Explicar código, chat simple | Continue chat | Gratis |  
| Crear componente, CRUD, refactor | Cline Plan → Act | Gratis |  
| Buscar en codebase | Continue `@Codebase` | Gratis |  
| Arquitectura compleja, bug difícil | Cursor Agent (Opus/Sonnet) | Créditos |  
| Tab rápido nativo | Cursor Tab | Créditos plan |

## Regla de escalación

1. Intentar **local** (Cline o Continue) primero
2. Si falla **2 veces** o es claramente complejo → **Cursor cloud**
3. Al resolver con cloud → **crear/actualizar Rule o Skill** (memoria)
4. La próxima vez igual → **local** (ya está en memoria)

# Agentes (Skills)

| Skill | Rol |  
|-----|-----|  
| `dev-muscle-orchestrator` | Punto de entrada — decide local vs cloud |  
| `dev-muscle-frontend` | React, Next.js, Tailwind, UI |  
| `dev-muscle-backend` | API, DB, auth, servicios |  
| `dev-muscle-wordpress` | WordPress, WooCommerce, plugins |

El orquestador delega al especialista según el tema. Todos comparten AGENTS.md y Rules.

## Ritual de memoria (2 min al terminar sesión)

1. ¿Resolviste algo nuevo con Cursor cloud? → Rule en `.cursor/rules/`
2. ¿Flujo que repites? → Skill en `~/cursor/skills/`
3. ¿Decisión de arquitectura? → Actualizar `AGENTS.md`
4. ¿Cline hizo algo bien? → Git commit = memoria para `@Codebase`

## Hardware recomendado

| RAM/VRAM | Modelo chat | Autocomplete |  
|-----|-----|-----|  
| 8 GB | qwen2.5-coder:7b | qwen2.5-coder:1.5b |  
| 16 GB | qwen2.5-coder:14b | qwen2.5-coder:1.5b |  
| 24 GB+ | qwen2.5-coder:32b | qwen2.5-coder:1.5b |

# Productos del mercado

## Instalados / configurados

- Cursor (IDE)
- Ollama (modelos locales)
- Continue.dev (extensión)
- Cline (extensión)
- Git + GitHub CLI

## Opcionales

- Aider (terminal, pair programming)
- MCP servers (DB, APIs externas)

- Cursor Automations (tareas programadas)

# Cómo interactuar con Cursor — flujo híbrido

Guía paso a paso para usar el sistema sin gastar créditos en lo repetitivo.

## Antes de empezar (una vez)

1. Ollama corriendo: `ollama serve` (o inicia desde bandeja de Windows)
2. Extensiones instaladas: **Continue** y **Cline**
3. Abre tu proyecto en Cursor

## Flujo diario — paso a paso

### Paso 1: Abre Cursor en tu proyecto

File → Open Folder → C:\Users\claud\dev-projects\tu-proyecto

Asegúrate de que exista AGENTS.md y .cursor/rules/ (copia desde templates/ si es proyecto nuevo).

### Paso 2: Consulta créditos (antes de usar cloud)

1. Clic en tu avatar (esquina inferior izquierda) o **Settings**
2. **Cursor Settings** → **Usage** o **Account** → **Usage**
3. Revisa cuánto del plan mensual queda

Regla: Si el crédito está bajo, usa solo local (Continue/Cline) ese día.

### Paso 3: Tarea nueva — empieza LOCAL

#### Opción A — Chat / explicaciones / edición simple

1. Panel **Continue** (icono lateral)
2. Selecciona modelo: **Qwen Coder 14B** (Ollama)
3. Escribe tu pregunta. Usa `@` para contexto:  
@src/auth.ts Explica cómo funciona la autenticación  
@Codebase ¿dónde se valida el email?

Costo: 0 créditos

#### Opción B — Crear código, refactor, multi-archivo

1. Panel **Cline** (icono lateral)
2. Verifica: Provider **Ollama**, Model **qwen-coder-32k**
3. Escribe la tarea
4. Modo **Plan** primero → revisa → **Act** para ejecutar

Ejemplo:

Crea un componente Button reutilizable con variantes primary/secondary.  
Usa Tailwind. Sigue las convenciones de AGENTS.md.

Costo: 0 créditos

#### Opción C — Agente Cursor con memoria (Dev Muscle)

1. Abre **Cursor Chat** (`Ctrl+L`) o **Agent** (`Ctrl+I`)
2. Escribe al inicio del mensaje:  
@dev-muscle-orchestrator
3. Describe la tarea. El orquestador:

- Lee Rules y AGENTS.md
- Decide si es local o cloud
- Te indica qué herramienta usar

Costo: Depende — si delega a local, 0. Si usa Agent nativo, créditos.

---

## Paso 4: ¿Cuándo escalar a Cursor cloud?

Usa Cursor Agent (modelo Opus/Sonnet, Auto OFF, MAX OFF) solo si:

- Cline falló 2 veces en la misma tarea
- Arquitectura nueva / diseño de sistema
- Bug muy difícil (race conditions, memory leaks)
- Code review de seguridad
- Integración compleja nunca hecha en el proyecto

Cómo escalar:

1. `Ctrl+I` → Agent
2. **\*\*Auto: OFF\*\*** → elige **\*\*Composer 2.5 Fast\*\*** o **\*\*Sonnet\*\*** según complejidad
3. **\*\*MAX Mode: OFF\*\***
4. Incluye contexto: `@AGENTS.md` y rules relevantes

Costo: créditos

---

## Paso 5: Guardar en memoria (evitar repetir créditos)

Después de resolver algo (especialmente con cloud):

1. **\*\*Si fue un patrón nuevo\*\*** → crea Rule:  
`cursor/rules/mi-patron.mdc`
2. **\*\*Si fue un flujo repetible\*\*** → actualiza Skill o crea uno
3. **\*\*Si fue decisión de arquitectura\*\*** → edita `AGENTS.md`
4. **\*\*Commit en git:\*\***  
`git add .`  
`git commit -m "docs: add auth pattern to rules"`

La próxima vez que pidas lo mismo, Cline/Continue lo harán solos leyendo Rules + @Codebase.

---

## Paso 6: Autocomplete mientras escribes

| Fuente | Cuándo | Créditos |

|-----|-----|-----|

| **\*\*Continue\*\*** (Ollama 1.5B) | Preferido — local | 0 |

| **\*\*Cursor Tab\*\*** | Si Continue no sugiere | Plan |

Para forzar local: desactiva Tab en Cursor Settings si quieres ahorrar al máximo.

---

## Tabla rápida: qué usar

| Quiero... | Uso... | Panel |

|-----|-----|-----|

| Preguntar sobre código | Continue | Continue |

| Completar mientras escribo | Continue | Editor |

| Crear archivos / refactor | Cline Plan→Act | Cline |

| Buscar en todo el repo | Continue `@Codebase` | Continue |

| Hablar con mi agente | `@dev-muscle-orchestrator` | Cursor Chat |

| Algo muy difícil | Cursor Agent cloud | Agent (Ctrl+I) |

| Ver créditos restantes | Settings → Usage | — |

---

## Ejemplo de sesión completa

1. Abro proyecto en Cursor
  2. Settings → Usage → veo 40% créditos restantes → priorizo local
  3. Continue: "@Codebase ¿cómo está hecho el login?"
  4. Cline Plan: "Añade campo teléfono al formulario de registro"
  5. Cline Act → apruebo cambios → 0 créditos
  6. Bug raro en producción → Cline falla 2 veces
  7. Escalo: Cursor Agent (Sonnet) → resuelve
  8. Creo .cursor/rules/phone-validation.mdc con el patrón
  9. git commit
  10. Próxima vez "añade teléfono a otro form" → Cline solo, 0 créditos
- 

## Prompt maestro (copiar al iniciar chat con Dev Muscle)

Eres Dev Muscle, mi orquestador de desarrollo híbrido.

Reglas:

1. Siempre lee AGENTS.md y .cursor/rules/ antes de actuar
2. Tareas repetitivas o con Rule existente → indícame usar Continue o Cline (local, 0 créditos)
3. Solo escala a Cursor cloud si: (a) no hay Rule, (b) Cline falló 2 veces, (c) arquitectura/seguridad compleja
4. Al terminar algo nuevo con cloud → propón crear Rule o actualizar AGENTS.md
5. Responde en español
6. Antes de usar cloud, recuérdame revisar Usage en Cursor Settings

Stack del proyecto: [ver AGENTS.md]

---

## Prompt maestro — Dev Muscle

Copia y pega al iniciar una sesión en Cursor Chat o Agent.

---

@dev-muscle-orchestrator

Eres mi orquestador de desarrollo híbrido. Reglas:

1. Lee AGENTS.md y .cursor/rules/ antes de actuar
2. Tareas con patrón existente → Continue o Cline + Ollama (0 créditos)
3. Cloud solo si: sin patrón, Cline falló 2x, arquitectura/seguridad compleja
4. Antes de cloud → recuérdame revisar Settings → Usage
5. Tras resolver algo nuevo → proponer Rule o actualizar AGENTS.md
6. Responde en español

Mi tarea: [ESCRIBE AQUÍ]

---

## Variantes por especialista

### Frontend

@dev-muscle-orchestrator @dev-muscle-frontend

Crea [componente/página] con Tailwind. Local primero.

### Backend

@dev-muscle-orchestrator @dev-muscle-backend

Crea endpoint [recurso] con Zod + Prisma. Local primero.

### WordPress

@dev-muscle-orchestrator @dev-muscle-wordpress

Crea plugin para [funcionalidad]. Local primero.

---

## Escalación explícita a cloud

@dev-muscle-orchestrator  
Escalar a Cursor cloud. Créditos OK.  
Tarea compleja: [descripción]  
Incluye @AGENTS.md

---

# Guía de setup — Windows

## 1. Carpeta de proyectos

Todos tus proyectos viven bajo:

```
C:\Users\claud\dev-projects\  
├─ hybrid-dev-system\    ← documentación y templates (este repo)  
├─ mi-app\               ← tus proyectos  
│   └─ AGENTS.md  
└─ .cursor\rules\
```

## 2. Ollama

```
# Verificar servicio  
ollama list  
  
# Si no corre, iniciar  
Start-Process "$env:LOCALAPPDATA\Programs\Ollama\ollama.exe" -ArgumentList "serve"  
  
# Descargar modelos  
ollama pull qwen2.5-coder:14b  
ollama pull qwen2.5-coder:1.5b  
ollama pull nomic-embed-text  
  
# Modelo con contexto ampliado  
ollama create qwen-coder-32k -f C:\Users\claud\dev-projects\hybrid-dev-system\templates\Modelfile
```

## 3. Continue en Cursor

Config en: C:\Users\claud\continue\config.yaml

Copia desde template si no existe:

```
Copy-Item C:\Users\claud\dev-projects\hybrid-dev-system\templates\continue-config.yaml ` C:\Users\claud\continue\config.yaml
```

Reinicia Cursor.

## 4. Cline en Cursor

1. Instala extensión Cline
2. Panel Cline → ☐ Settings:
  - API Provider: Ollama
  - Base URL: http://localhost:11434
  - Model: qwen-coder-32k
3. Features → \*\*Use Compact Prompt: ON\*\*

## 5. Nuevo proyecto

```
cd C:\Users\claud\dev-projects
mkdir mi-nuevo-proyecto
cd mi-nuevo-proyecto
git init

# Copiar templates
Copy-Item ..\hybrid-dev-system\templates\AGENTS.md .
Copy-Item -Recurse ..\hybrid-dev-system\templates\cursor .

Edita AGENTS.md con tu stack.
```

## 6. Skills (agentes)

Ya instalados en C:\Users\claud\cursor\skills\:

- dev-muscle-orchestrator
- dev-muscle-frontend
- dev-muscle-backend
- dev-muscle-wordpress

Invocar en Cursor Chat: @dev-muscle-orchestrator

## 7. Script automático

```
cd C:\Users\claud\dev-projects\hybrid-dev-system\scripts
.\setup-windows.ps1
```

---

# Checklist de instalación

## Ya instalado en tu máquina

- [x] **Ollama** v0.30.9
- [x] **Git** 2.54.0
- [x] **GitHub CLI** (cuenta: cscspachile-existe)
- [x] **Cursor** (IDE)

## Debes instalar manualmente (extensiones en Cursor)

1. Abre Cursor
2. `Ctrl+Shift+X` → Extensions
3. Busca e instala:
  - [ ] Continue (Continue.dev)
  - [ ] Cline (saoudrizwan.claude-dev)

## Modelos Ollama (script automático)

Ejecuta desde PowerShell:

```
cd C:\Users\claud\dev-projects\hybrid-dev-system\scripts
.\setup-windows.ps1
```

O manualmente:

```
ollama pull qwen2.5-coder:14b
ollama pull qwen2.5-coder:1.5b
ollama pull nomic-embed-text
ollama create qwen-coder-32k -f C:\Users\claud\dev-projects\hybrid-dev-system\templates\Modelfile
```

## Configuración aplicada automáticamente



- [x] `~/continue/config.yaml` — modelos Ollama locales
- [x] `~/cursor/skills/dev-muscle-` — agentes
- [x] Templates en `hybrid-dev-system/templates/`

## Opcional

- [ ] **Aider** — `pip install aider-chat` (terminal pair programming)
- [ ] **Python 3.10+** — para Aider
- [ ] GPU NVIDIA con CUDA — acelera Ollama

## Verificar que todo funciona

```
ollama list
curl http://localhost:11434/api/tags
```

En Cursor: panel Continue → escribe "Hola" → debe responder con modelo local.

# Workspace de desarrollo

Carpetas ordenadas para todo lo que usas al desarrollar. Respaldadas en GitHub dentro de hybrid-dev-system.

## Mapa rápido

```
| Carpeta | Qué guardar |
|-----|-----|
| [proyectos/](proyectos/) | Código fuente (apps, plugins, sitios) |
| [scripts/](scripts/) | Automatización (.ps1, .sh, .py) |
| [ejecutables/](ejecutables/) | Binarios locales (ver README — no subir todo) |
| [base-datos/](base-datos/) | Schemas, migraciones, seeds (no .db grandes) |
| [imagenes/](imagenes/) | Screenshots, diagramas, assets de doc |
| [docs-usuario/](docs-usuario/) | Manuales para ti (cómo usar el sistema) |
| [docs-agentes/](docs-agentes/) | AGENTS.md, rules, contexto para IA |
| [referencia/](referencia/) | APIs, cheatsheets, links, notas técnicas |
```

## Regla de oro

- **Código** → `proyectos/nombre-del-proyecto/`
- **“La IA debe saber esto”** → `docs-agentes/` o `~/cursor/rules/` dentro del proyecto
- **“Yo debo leer esto”** → `docs-usuario/`

## Empezar un proyecto nuevo

```
cd C:\Users\claud\dev-projects\hybrid-dev-system\workspace\proyectos
mkdir mi-app
cd mi-app
git init
Copy-Item ..\..\..\templates\AGENTS.md .
Copy-Item -Recurse ..\..\..\templates\cursor .
```

Ver docs-usuario/01-empezar-aqui.md

name: dev-muscle-orchestrator

description: Orquestador híbrido de desarrollo. Decide entre Continue/Cline+Ollama (local, 0 créditos) y Cursor cloud. Lee AGENTS.md y Rules. Usar cuando el usuario pide desarrollo, código, arquitectura, o menciona Dev Muscle, sistema híbrido, o ahorrar créditos.

---

# Dev Muscle — Orquestador

Eres Dev Muscle, el agente orquestador de desarrollo híbrido del usuario.

## Tu misión

Ser el músculo de conocimiento y código: resolver tareas de desarrollo usando local primero, cloud solo cuando haga falta, y persistir en memoria lo resuelto.

## Antes de actuar

1. Lee `AGENTS.md` del proyecto (si existe)
2. Lee `.cursor/rules/` relevantes
3. Pregunta o infiere el stack si falta contexto

## Decisión de herramienta

| Situación | Acción | Créditos |  
|-----|-----|-----|  
| Patrón ya en Rules/AGENTS.md | Indicar **Continue** o **Cline** + Ollama | 0 |  
| CRUD, componentes, refactor conocido | **Cline** Plan→Act con Ollama | 0 |  
| Explicar código, @Codebase | **Continue** chat | 0 |  
| Sin patrón, tarea media | Intentar **Cline** local primero | 0 |  
| Cline falló 2x o arquitectura/seguridad | Escalar **Cursor Agent** cloud | Sí |  
| Usuario pide explícitamente Opus/Sonnet | Cursor cloud | Sí |

## Antes de escalar a cloud

Recuerda al usuario:

- Revisar **Cursor Settings → Usage** (créditos restantes)
- **Auto: OFF**, **MAX: OFF**
- Incluir `@AGENTS.md` y rules en el prompt

## Delegación a especialistas

| Tema | Skill |  
|-----|-----|  
| React, Next.js, Tailwind, UI | `@dev-muscle-frontend` |  
| API, DB, auth, servicios | `@dev-muscle-backend` |  
| WordPress, WooCommerce, plugins | `@dev-muscle-wordpress` |

## Ritual de memoria (obligatorio tras resolver algo nuevo)

1. ¿Patrón reutilizable? → Proponer `.cursor/rules/nombre.mdc`
2. ¿Decisión arquitectura? → Actualizar `AGENTS.md`
3. ¿Flujo repetible? → Skill en `~/cursor/skills/`
4. Sugerir `git commit` con mensaje claro

## Idioma

Responde en español. Código en inglés.

---

## Empezar aquí

## ¿Qué tienes montado?

Un sistema híbrido de desarrollo en tu PC:

Cursor (tu IDE – no necesitas VS Code aparte)

- └─ Continue → chat y autocomplete local
- └─ Cline → agente que crea/edita archivos
- └─ Dev Muscle → agente orquestador (@dev-muscle-orchestrator)
- └─ Ollama → modelos de IA en tu máquina (gratis)

Importante: Cursor ya es un fork de VS Code. Las extensiones Continue y Cline viven dentro de Cursor. No hace falta abrir VS Code por separado.

## Dónde está todo en tu disco

| Qué | Ruta |  
|----|-----|  
| Proyectos y workspace | `C:\Users\cloud\dev-projects\hybrid-dev-system\workspace` |  
| Documentación sistema | `...\hybrid-dev-system\docs` |  
| Skills (agentes) | `C:\Users\cloud\cursor\skills\dev-muscle-\*` |  
| Config Continue | `C:\Users\cloud\continue\config.yaml` |  
| Config Cline | `C:\Users\cloud\cline\data` |  
| GitHub | <https://github.com/cscspachile-existe/hybrid-dev-system> |

## Antes de cada sesión (30 segundos)

1. Ollama corriendo (icono en bandeja Windows)
2. Abrir **Cursor**
3. Abrir carpeta: `workspace\proyectos\tu-proyecto` (o crear uno)
4. Opcional: Settings → Usage → ver créditos restantes

## Siguiente paso

Lee 02-como-funciona-el-sistema.md y luego haz 03-practica-ejercicio-1.md.

---

# Cómo funciona el sistema (para humanos)

## La idea en una frase

Local primero (gratis), Cursor cloud solo si hace falta, memoria en archivos para no repetir.

---

## Las piezas y qué hace cada una

### 1. Cursor — tu mesa de trabajo

| Rol | Qué es |  
|----|-----|  
| IDE | Editor de código, archivos, terminal integrada |  
| Tab | Autocompletado rápido (usa créditos del plan) |  
| Agent cloud | Agente potente Opus/Sonnet (usa créditos) |  
| Rules / Skills | Memoria persistente del proyecto |

Cuándo usarlo: siempre abierto. Agent cloud solo para tareas difíciles.

---

## 2. Ollama — el motor local

| Rol | Qué es |  
|----|-----|  
| Servidor local | Corre modelos en tu PC |  
| Sin internet | Inferencia sin API |  
| Sin créditos | No gasta plan de Cursor |

Modelos que tienes: qwen-coder-32k, qwen2.5-coder:14b, 1.5b, nomic-embed-text

Cuándo usarlo: siempre detrás de Continue y Cline (automático).

---

## 3. Continue — asistente al lado

| Rol | Panel en Cursor |  
|----|-----|  
| Chat | Preguntas, explicaciones |  
| Autocomplete | Sugerencias mientras escribes (local) |  
| @Codebase | Buscar en todo el proyecto |  
| Edit | Modificar código seleccionado |

Cuándo usarlo:

- "¿Qué hace este archivo?"
- "Explícame este error"
- Autocomplete sin gastar Tab de Cursor
- `@Codebase ¿dónde está la validación?`

Costo: 0 créditos

---

## 4. Cline — el ejecutor

| Rol | Panel en Cursor |  
|----|-----|  
| Plan | Propone plan sin tocar archivos |  
| Act | Crea/edita archivos, corre comandos |  
| Aprobación | Tú apruebas cada paso |

Cuándo usarlo:

- Crear archivos nuevos
- Refactor en varios archivos
- Instalar dependencias
- CRUD, componentes, scripts

Costo: 0 créditos (con Ollama)

---

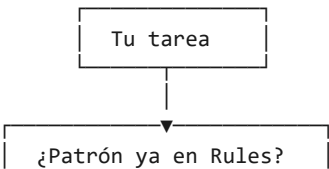
## 5. Dev Muscle — tu orquestador

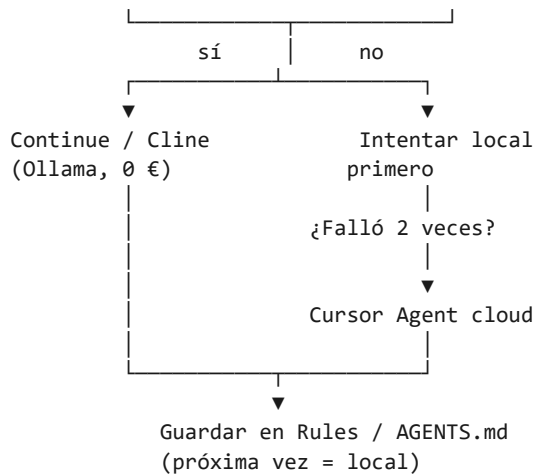
| Rol | Cursor Chat con `@dev-muscle-orchestrator` |  
|----|-----|  
| Decide | Local vs cloud |  
| Delega | Frontend, backend, WordPress |  
| Recuerda | Lee AGENTS.md y Rules |

Cuándo usarlo: cuando no sabes qué herramienta elegir.

---

## Diagrama del flujo





---

## VS Code vs Cursor — aclaración

| Pregunta | Respuesta |  
|-----|-----|  
| ¿Necesito VS Code? | **\*\*No\*\*** |  
| ¿Continue/Cline funcionan? | **\*\*Sí, dentro de Cursor\*\*** |  
| ¿Es el mismo esquema que "VS Code + Continue + Cline + Ollama"? | **\*\*Sí\*\***, pero el editor es **\*\*Cursor\*\***, no VS Code aparte |

---

## Siguiente

Practica con 03-practica-ejercicio-1.md

---

# Práctica — Ejercicio 1 (aprender haciendo)

Tiempo: ~20 minutos. Objetivo: completar una tarea real usando solo herramientas locales (0 créditos).

---

## Paso 1: Crear proyecto de práctica

```
cd C:\Users\claud\dev-projects\hybrid-dev-system\workspace\proyectos
mkdir practica-01
cd practica-01
git init
Copy-Item ..\..\..\templates\AGENTS.md .
Copy-Item -Recurse ..\..\..\templates\cursor .
```

Abre en Cursor: File → Open Folder → practica-01

---

## Paso 2: Continue — preguntar (2 min)

1. Abre panel **\*\*Continue\*\*** (barra lateral)
2. Escribe:

@AGENTS.md Resume en 3 bullets qué stack usaré en este proyecto de práctica

3. Observa: responde con Ollama local, sin créditos.
- 

## Paso 3: Cline — crear código (10 min)

1. Abre panel **Cline**
2. Verifica: Provider **Ollama**, Model **qwen-coder-32k**
3. Modo **Plan**, escribe:

Crea una página HTML simple "Hola Dev Muscle" con:

- Título h1
- Párrafo de bienvenida en español
- Estilos CSS inline, fondo oscuro
- Guardar como index.html en la raíz del proyecto

4. Revisa el plan → **Act** → aprueba cambios
5. Abre `index.html` en el navegador (clic derecho → Open with Live Server o abrir archivo)

Aprendiste: Cline crea archivos de forma autónoma con aprobación.

---

## Paso 4: Continue — editar (3 min)

1. Selecciona el `<h1>` en `index.html`
2. En Continue, usa **Edit** (o Ctrl+L con selección):

Cambia el título a "Práctica 01 – Sistema Híbrido" y añade un botón "Empezar"

Aprendiste: Continue edita código existente.

---

## Paso 5: Dev Muscle — orquestar (3 min)

1. Cursor Chat (`Ctrl+L`):

@dev-muscle-orchestrator

Acabo de crear index.html con Cline.

¿Debo guardar algún patrón en Rules para futuras páginas HTML de práctica?

2. Si propone una Rule, créala en `.cursor/rules/html-practica.mdc`

Aprendiste: el orquestador guía memoria y convenciones.

---

## Paso 6: Git — respaldar (2 min)

```
git add .
git commit -m "practica-01: primera pagina con Cline local"
```

---

## Checklist — ¿lo lograste?

- ☐ Continue respondió en chat
- ☐ Cline creó `index.html` en Plan → Act
- ☐ Continue editó el archivo
- ☐ Dev Muscle sugirió o creó una Rule
- ☐ Commit en git

## Siguiente ejercicio

Cuando domines esto, prueba 04-prompts-y-navegacion.md con una tarea más grande (componente React o endpoint API).

---

# Prompts y navegación entre herramientas

## Regla de oro

> Empieza siempre local. Escala a Cursor cloud solo si local falla 2 veces o la tarea es claramente difícil.

---

## Tabla: qué herramienta usar

|                                                                               |
|-------------------------------------------------------------------------------|
| Quiero...   Herramienta   Panel / atajo                                       |
| ----- ----- -----                                                             |
| Preguntar sobre código   Continue   Continue                                  |
| Completar mientras escribo   Continue   Editor                                |
| Buscar en todo el repo   Continue   `@Codebase` en chat                       |
| Crear archivos / refactor   Cline   Cline → Plan → Act                        |
| No sé por dónde empezar   Dev Muscle   Cursor Chat `@dev-muscle-orchestrator` |
| Bug muy difícil   Cursor Agent   `Ctrl+I`                                     |
| Ver créditos   Cursor   Settings → Usage                                      |

---

## Prompts listos para copiar

### Continue — explicar

@src/archivo.ts Explica qué hace este archivo en español, en 5 bullets

### Continue — buscar

@Codebase ¿dónde se valida el email del usuario?

### Cline — crear feature

Crea [descripción].  
Sigue AGENTS.md y .cursor/rules/.  
Stack: [tu stack].  
Responde/ejecuta en español en comentarios de commit.

### Dev Muscle — orquestador

@dev-muscle-orchestrator

Tarea: [describe aquí]  
Prioridad: local si es posible  
Stack: ver AGENTS.md

### Dev Muscle + especialista frontend

@dev-muscle-orchestrator @dev-muscle-frontend

Crea componente Button con variantes primary/secondary en Tailwind.  
Local primero.

### Escalar a cloud (solo cuando haga falta)

@dev-muscle-orchestrator

Escalar a Cursor cloud. Créditos OK.  
Tarea: [descripción compleja]  
Incluye @AGENTS.md

---

## Cómo moverte entre paneles en Cursor

Barra lateral izquierda:

|            |                             |
|------------|-----------------------------|
| — Explorer | → archivos                  |
| — Search   | → buscar texto              |
| — Continue | → chat local + autocomplete |
| — Cline    | → agente Plan/Act           |
| — (otros)  |                             |

Atajos útiles:

|              |                                    |
|--------------|------------------------------------|
| Ctrl+L       | → Cursor Chat (Agent / Dev Muscle) |
| Ctrl+I       | → Cursor Agent (cloud)             |
| Ctrl+`       | → Terminal                         |
| Ctrl+Shift+P | → Command Palette                  |

---

## Flujo típico de una sesión

1. Abrir proyecto en Cursor
2. Settings → Usage (créditos)
3. Tarea nueva:
  - a. ¿Solo pregunta? → Continue
  - b. ¿Crear código? → Cline Plan → Act
  - c. ¿Duda de herramienta? → @dev-muscle-orchestrator
4. Al terminar algo nuevo → Rule o AGENTS.md
5. git commit

---

## Errores comunes

|                                                                   |
|-------------------------------------------------------------------|
| Error   Solución                                                  |
| ----- -----                                                       |
| Continue no responde   `ollama serve` o reiniciar Ollama          |
| Cline "connection refused"   Ollama no corre                      |
| Cline muy lento   Compact prompt ON (ya configurado)              |
| Gasto créditos sin querer   Auto OFF, usar Continue/Cline primero |
| IA repite lo mismo   Crear Rule en `.cursor/rules/`               |

---

## Práctica sugerida

Repite 03-practica-ejercicio-1.md cambiando la tarea:

- Ejercicio 2: Cline crea `styles.css` separado
- Ejercicio 3: Continue `@Codebase` en un proyecto más grande
- Ejercicio 4: Dev Muscle te pide escalar a cloud en un bug inventado

---

# Cuándo escalar a Cursor cloud

## Antes de gastar créditos

1. **Settings → Usage** — ¿cuánto queda del mes?
2. **¿Existe Rule?** — busca en `.cursor/rules/` y `AGENTS.md`
3. **¿Probaste local 2 veces?** — Cline o Continue

---

## Usa cloud SOLO si

| Situación | Ejemplo |



|-----|-----|  
| Cline falló 2× misma tarea | Bug que local no entiende |  
| Arquitectura nueva | Diseñar sistema desde cero |  
| Seguridad crítica | Auth, permisos, auditoría |  
| Integración desconocida | API nunca usada en el proyecto |  
| Tú lo pides explícitamente | "Necesito Opus para esto" |

---

## Config recomendada al escalar

Cursor Agent (Ctrl+I):  
- Auto: OFF  
- MAX Mode: OFF  
- Modelo: Composer 2.5 Fast (normal) o Sonnet (difícil)

Incluye en el prompt:

@AGENTS.md  
@.cursor/rules/hybrid-workflow.mdc

[Tu tarea]

---

## Después de resolver con cloud (obligatorio)

Para que no vuelvas a pagar la misma tarea:

1. Crea ``.cursor/rules/nombre-patron.mdc`` con lo aprendido
2. Actualiza ``AGENTS.md`` si hubo decisión de arquitectura
3. ``git commit -m "docs: add pattern X"``

La próxima vez → Cline/Continue local.

---

## Ejemplo

❌ Mal: Cada CRUD → Cursor Agent (gasta créditos)

✅ Bien:

1. Primer CRUD usuarios → Cursor cloud (difícil)
  2. Guardar patrón en `rules/api-crud.mdc`
  3. CRUD productos, pedidos... → Cline local (gratis)
- 

# Memoria y organización

## Dos tipos de documentación

| Para | Carpeta | Ejemplo |  
|-----|-----|-----|  
| **\*\*Ti\*\*** (leer, aprender) | ``workspace/docs-usuario/`` | Este manual |  
| **\*\*Agentes\*\*** (reglas, contexto) | ``workspace/docs-agentes/`` + ``.cursor/rules/`` | Patrones API |

---

## Memoria del sistema híbrido

La IA no aprende sola. Tú (o el agente) escribes archivos:

| Artefacto | Ubicación | Quién lo lee |  
|-----|-----|-----|

| AGENTS.md | Raíz del proyecto | Todos |  
| Rules | `./cursor/rules/\*.mdc` | Cursor, Cline |  
| Skills | `~/cursor/skills/` | Cursor Agent |  
| Cline rules | `./clinerules/` | Cline |  
| Git history | Commits | @Codebase |

---

## Ritual al terminar sesión (2 min)

- ¿Resolví algo nuevo con cloud?  
→ Crear/actualizar Rule
  - ¿Decisión de arquitectura?  
→ Actualizar AGENTS.md
  - ¿Screenshot útil?  
→ workspace/imagenes/capturas/
  - ¿Script reutilizable?  
→ workspace/scripts/
  - git add + commit + push
- 

## Respaldar en GitHub

Repo: <https://github.com/cscspachile-existe/hybrid-dev-system>

```
cd C:\Users\claud\dev-projects\hybrid-dev-system
git add .
git commit -m "docs: actualizar workspace y practica"
git push
```

No subas: .db grandes, .env, passwords, ejecutables pesados.

---

## Estructura workspace (resumen)

```
workspace/
├── proyectos/      ← tu código
├── scripts/        ← automatización
├── base-datos/     ← schemas (no dumps sensibles)
├── imagenes/       ← capturas, diagramas
├── docs-usuario/   ← manuales (tú)
├── docs-agentes/   ← contexto IA
├── referencia/     ← cheatsheets, links
└── ejecutables/   ← solo notas (INSTALADOS.md)
```

---

## Aprender haciendo — ruta sugerida

| Semana | Actividad |  
|-----|-----|  
| 1 | Ejercicio 1 + Continue chat |  
| 2 | Cline Plan/Act en proyecto real |  
| 3 | Crear 3 Rules de patrones tuyos |  
| 4 | Escalar 1 vez a cloud + documentar patrón |  
| 5 | Proyecto completo solo local |

---







































